Claude-Ökosystem im Überblick: Code, Cowork, Features, Hacks

Quellen: video-summary-sQBinJA_zxU.md ("Claude Code vs. Claude Co-Work"), video-summary-B_OqkMRFonM.md ("Jede Claude Funktion erklärt"), video-summary-wZeOwqmSw84.md ("Learn 97% of Claude in Under 16 Minutes"). Praxistipps-Ergänzung: video-summary-KZAJeq5n-m8.md, video-summary-VKMNP_5vOmM.md, video-summary-XEbR5qmxGQ0.md, video-summary-W30OHZdd7WQ.md, video-summary-LoMOPj-lO8U.md, video-summary-IYzgxWs4sZ4.md, video-summary-JDebq_fxLlw.md, video-summary-zB9it8-nbeM.md, video-summary-v_USOf1fSYM.md (Nachtrag 2026-10-24, Chat/Cowork/Code-Überblick), video-summary-iVxubkOM4gA.md (Nachtrag 2026-10-24, Scion/Finanzen)
~~Quellen: video-summary-sQBinJA_zxU.md ("Claude Code vs. Claude Co-Work"), video-summary-B_OqkMRFonM.md ("Jede Claude Funktion erklärt"), video-summary-wZeOwqmSw84.md ("Learn 97% of Claude in Under 16 Minutes"). Praxistipps-Ergänzung: video-summary-KZAJeq5n-m8.md, video-summary-VKMNP_5vOmM.md, video-summary-XEbR5qmxGQ0.md, video-summary-W30OHZdd7WQ.md, video-summary-LoMOPj-lO8U.md, video-summary-IYzgxWs4sZ4.md, video-summary-JDebq_fxLlw.md, video-summary-zB9it8-nbeM.md~~

Drei Videos decken sich stark in ihrem Ziel — einen vollständigen Überblick über Claudes Funktionsumfang zu geben — nutzen aber unterschiedliche Gliederungen. Hier zusammengeführt statt dreifach redundant.

Die zwei Philosophien: Claude Code vs. Claude Cowork

Aus sQBinJA_zxU, mit der treffendsten Kernaussage: **"Code ist der Maschinenraum, Cowork die Chefetage."**

- **Claude Code** — Terminal-basiert, voller Systemzugriff, für tiefe technische Eingriffe (Datenbanken aufsetzen, Git, Netzwerk). Sehr token-effizient, da keine grafische Oberfläche geladen wird — relevant, weil ein Agent-Loop teils 50-100 Iterationen brauchen kann.
- **Claude Cowork** — nicht zum Software-Bauen gedacht, sondern zur Orchestrierung von Alltags-Workflows (Mail, Kalender, Dateisortierung). Hat persistentes Gedächtnis über Sessions hinweg.

Der zugrundeliegende Mechanismus in beiden ist der **Agent Loop**: Kontextualisierung → Planung → Ausführung → Validierung.

Die Bauteile, geordnet (Roboter-Metapher aus B_OqkMRFonM)

Arbeitsplatz — Workspace-Ordner, Claude.md (projektspezifisch), globale Anweisungen (app-weit), Projects (Ordner mit eigenem Memory/Kontext)

Gehirn — Memory (Langzeitgedächtnis, automatisch + manuell), Kontextfenster (Kurzzeitgedächtnis, für neue große Aufgaben lieber neuen Chat starten), Extended Thinking (mehr Nachdenkzeit gegen mehr Nutzungskontingent)

Augen/Sensorik — Multimodal (Screenshots, PDFs, Tabellen; kein Video laut Video), Websuche, Recherche (Deep-Research-artige Reports mit Quellen), Connectors (Gmail, Calendar, Notion, Slack), MCP (die offene Schnittstelle dahinter — "der USB-Anschluss für KI-Tools", ausführlich in mcp-ueberblick.md)

Hände — Artifacts (interaktive Mini-Apps im Chat), Chrome-Erweiterung (Browser-Steuerung), Computer Use (volle Rechnersteuerung, laut beiden Videos noch nicht sehr schnell)

Autopilot — Skills (wiederverwendbare Arbeitsabläufe, laut B_OqkMRFonM "der wichtigste Punkt"), Slash-Commands (Skill-Trigger), Plugins (Skill-Bündel), Scheduled Tasks/Automations, Dispatch Mode (Mobile → Desktop-Steuerung)

Ergänzende Hacks aus wZeOwqmSw84 (nicht in den anderen beiden enthalten)

- **Memory Import** — ChatGPT-Verlauf/Präferenzen exportieren und in Claude importieren
- **Model Selector** — Haiku (schnell/günstig) vs. Sonnet (Alltag) vs. Opus (komplexe Analysen), alternativ "Adaptive Thinking"
- **Voice Mode** — Diktieren statt Tippen
- **Claude Channels** — Anbindung an iMessage/Telegram/Discord (nicht gegengecheckt)
- **Claude Design** (claude.ai/design) — Pitch-Decks, Landingpages, Mockups, Motion Graphics, mit Adobe-Partnerschaft (nicht gegengecheckt)

Praktische Einordnung

Skills sind laut sQBinJA_zxU technisch mehr als Custom Instructions: echte lokale Ordner mit Markdown + Skripten. Das Kontextfenster-Problem wird über **Progressive Disclosure** gelöst — nur die per semantischer Suche passende Skill-Datei wird geladen, nicht das ganze "Firmenhandbuch" auf einmal. Ausführliches Grundkonzept inkl. Beispielen und Bauanleitung: claude-skills-ueberblick.md.

Praxistipps aus weiteren Videos

Neun zusätzliche Videos liefern konkrete, im Kern verifizierbare Praxistipps zum Alltag mit Claude Code — ergänzend zum Grundkonzept oben:

- **Kostenoptimierung** (video-summary-KZAJeq5n-m8.md, video-summary-XEbR5qmxGQ0.md): Kontextfenster-Sichtbarkeit (Status Line, `/context`), Prompt-Caching-Mechanik (Rabatt verfällt bei Modellwechsel, Reasoning-Level-Wechsel oder Inaktivität), Subagent-Orchestrierung, Handover-Dateien statt automatischer Compaction statt Mikro-Optimierungen

- **Output-Styles** (video-summary-VKMNP_5vOmM.md): Gegen Opus 5s Hang zu verschachtelten, jargonlastigen Antworten helfen selbst definierte Styles (`.claude/settings.local.json`), testbar per `/rewind` und `Branch`
- **Effort-Stufe als Kostenhebel** (video-summary-zB9it8-nbeM.md): Fable 5.1 führt auf höchsten Effort-Einstellungen (X-High/Max) bei sehr hohem Token-Verbrauch, ist auf mittleren Einstellungen aber kostenmäßig konkurrenzfähig zu GPT-5.6 Sol/Grok 4.6 — die Effort-Wahl entscheidet über die Kosten, nicht das Modell an sich
- **Reasoning-Effort bei Coding** (video-summary-IYzgxWs4sZ4.md): Opus 5 performt bei Coding-Benchmarks teils *besser* mit Medium- statt Extra-High-Reasoning-Effort (Over-Engineering-Risiko bei zu viel Denkzeit)
- **Anthropic Academy** (video-summary-W30OHZdd7WQ.md): Kostenlose Kurse mit Zertifikat, Pflicht-Empfehlung für "AI Fluency: Frameworks and Foundations" (Kapitel Diligence), "Claude Code in Action" und "Introduction to Subagents"; API-Kurs und Cloud-Plattform-Kurse (Bedrock/Vertex) für die meisten optional
- **INTENT.md / AI-Native SDLC** (video-summary-LoMOPj-lO8U.md): Anthropics Blogpost formalisiert den Entwicklungszyklus als Kette versionierter Artefakte (intent.md → spec.md → plan.md → PR-Review → Maintenance-Loop) — ausdrücklich eine von mehreren validen Herangehensweisen, kein zwingender Ersatz für bestehende Workflows
- **MCP-Tool-Integration** (video-summary-JDebq_fxLlw.md): Higgsfield bündelt 30+ KI-Bild-/Videomodelle per MCP-Server direkt in Claude Code — Nutzen hängt am Credit-Verbrauch, "Unlimited"-Pläne greifen im Terminal-Workflow nicht

Prompting Claude 5: 7 Regeln (hEWqRbQe6z4)

Zusammenfassung des Opus-5-Prompting-Guides (Guide und Umgebungsvariablen per WebSearch bestätigt): ganze Aufgabe vorab nennen; Interview zuerst (Community-Technik, keine Anthropic-Regel); das "Warum" nennen; Definition of Done angeben; begründete statt starrer Regeln; alte Tricks (u. a. Selbstverifikation im Prompt) weglassen; Antwortstil zentral in der CLAUDE.md. Überschneidet sich mit ai-agent-workflow.md (Interview-Prompt) und context-rot-ueberblick.md (Systemprompt-Kürzung). Mögliche Spannung: Regel 6 gegen "immer gegenprüfen" — aufgelöst als "Selbstverifikation im Prompt ist redundant, menschliche Prüfung bleibt". Im Video genanntes "Haiku 5.1" ließ sich nicht bestätigen.

Nachtrag Faktencheck (2026-09-29): Die "7 Regeln" sind die Zählung des Hosts; im Guide belegt sind Verifikations-Anweisungen entfernen, Scope, Verbosity-Beispiel und Subagent-Limits (`CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH` Default 3, `CLAUDE_CODE_MAX_CONCURRENT_SUBAGENTS` Default 20). "Haiku 5.1" existiert vermutlich nicht (Fable 5.1 und Mythos 5.1 erschienen am 01.09.). Siehe video-summary-hEWqRbQe6z4.md.

Nachtrag 2026-10-24: Chat/Cowork/Code für Einsteiger, Mods und Anthropics Rekorde

Chat, Cowork und Code erklärt — was wozu gehört (v_USOf1fSYM)

Quelle: video-summary-v_USOf1fSYM.md ("CLAUDE kann jetzt ALLES?", Christoph Magnussen / Blackboard Academy, 24 Min). **Prüfstatus:** Mods (TypeScript) per Anthropic-Dokumentation und WebSearch bestätigt; Dashboards in Artifacts bestätigt; Design-Tab-Umzug Dezember 2026 nicht unabhängig verifiziert.

Das Video leistet, was keines der bisherigen Quellen dieses Artikels explizit tut: die Drei-Produkte-Logik für Laien erklären. Die Unterscheidung aus sQBinJA_zxU ("Code ist Maschinenraum, Cowork ist Chefetage") wird hier mit anderen Worten für ein breiteres Publikum aufgeschlüsselt:

- **Claude Chat** — für schnelle, einzelne Fragen; kein persistentes Gedächtnis über Sitzungen
- **Claude Cowork** — eigenständige Aufgaben über mehrere Schritte, läuft in Anthropics Cloud, hat persistentes Gedächtnis; der Host beschreibt es als "Pendant zu Google — nur dass Claude echte Antworten formuliert statt Links aufzulisten"
- **Claude Code** — läuft lokal auf dem eigenen Computer, voller Dateisystemzugriff, für technisch komplexe, langfristige Aufgaben; mit Unter-Agenten und `/goal`-Befehl für übergeordnete Ziele

Mods (Claude Code): Kleine TypeScript-Bausteine (laut Anthropic-Dokumentation, per WebSearch bestätigt; Anthropic 01.10.2026) verändern das Erscheinungsbild und Verhalten der Claude-Code-App selbst. Kein Sandboxing — Mods haben denselben Systemzugriff wie Claude Code. Das Video zeigt ein konkretes Beispiel: Bestätigungs-Prompt aus dem Textfluss in einen farbkodierten Klick-Button umgebaut. Nur aus vertrauenswürdigen Quellen installieren.

Plugins als Skills-plus-Connectors-Bundle: Bisher wurden Skills und Connectors einzeln verwaltet. Plugins bündeln beides — Connectors für Datenzugriff (CRM, Datenbank, Cloud) und Skills für Arbeitsabläufe — in einem installierbaren Paket. Aktivierung per `@PluginName` im Gespräch. Teams-Account ermöglicht zentrales Plugin-Management für alle Mitarbeitenden.

Design-System als Konfigurationsebene für Agenten: Das Video begründet, warum Design-Systeme mit mehr KI-Agenten wichtiger werden: Wer Agenten autonom Dokumente, Mails oder Webinhalte erstellen lässt, braucht klare Regeln für Farben, Tonalität und Layout — sonst entstehen konsistenzfreie Outputs. Design-Systeme können auf "Schreibsysteme" (Stil und Formulierungen) und "Verhaltensregeln für Agenten" (was darf ein Agent tun) ausgeweitet werden.

Modell-Strategie: Haiku 5.5 ist für Unter-Agenten in komplexen Workflows rehabilitiert — Haupt-Agent mit Opus 5.5, Teilaufgaben mit Haiku 5.5, spart erhebliche Kosten. In der CLAUDE.md kann das automatisch konfiguriert werden.

Anthropic bricht Entwicklungsrekorde — Scion ersetzt Git (iVxubkOM4gA)

Quelle: video-summary-iVxubkOM4gA.md (Everlast AI / Leonard Schweding, Oktober 2026). **Prüfstatus:** Scion-Meldung und Finanzzahlen per Reuters-Bericht bestätigt (Suchtreffer).

Scion ersetzt Git: Im Mai 2026 gab es ~2.500 Codeänderungen pro Tag bei Anthropic; im September >30.000 — eine Änderung alle drei Sekunden. Git konnte das nicht mehr bewältigen. Sieben Ingenieure bauten **Scion** (englisch "Sprössling/Ableger") als Ersatz: laut Anthropic-Angaben

~200-mal weniger Rechenleistung und Speicher als die bisherige Infrastruktur. Das Team bestätigt: Claude schreibt inzwischen besseren Code als die menschlichen Entwickler selbst.

Finanzen: Anthropics Quartalsumsatz stieg um ~50 % auf **$16,8 Mrd.** (Q3 2026). Das Unternehmen war in den vergangenen sechs Monaten auf adjusted-Basis bereits profitabel. Anthropic meldete vertraulich einen Börsengang und plant das Listing für **November 2026** (laut externen Berichten; im Video nicht datiert; Suchtreffer). Bewertungsband in Investorenkreisen: 1,5–2 Bio. $; das 2-Bio.-Ziel gilt als unter Druck, da der annualisierte Umsatz voraussichtlich unter 100 Mrd. $ bleibt.

Offene Fragen (nicht gegengecheckt)

- Genaue aktuelle Modellnamen/-versionen, wie in den Videos gezeigt (Screenshots sind Momentaufnahmen)
- Existenz/Funktionsweise von "Claude Dispatch", "Claude Channels" und "Claude Design" als eigenständige, benannte Produktfeatures — Videos könnten informelle Namen für tatsächliche Anthropic-Features verwenden
- Wissensstand-Cutoff "Mai 2025", den B_OqkMRFonM nennt
- Design-Tab-Umzug in Artifacts "Mitte Dezember 2026": aus Transkript, nicht extern bestätigt

Quelle: claude-oekosystem-ueberblick.md